# Getting data and scaling up

## Programming & Data Management, Part II (Python) · Session 4

Magnus Lodefalk
*Örebro University*

NA446A, Örebro University

Autumn 2026



# Today's goals

By the end of today you can:

1. Fetch data from an open API, save the raw answer, and check that every row arrived, as with Stata's `assert _N == ...`
2. Handle dates as with Stata's `monthly()`, `collapse` and `L4.`; write a repeated step once, as Stata's `program`
3. Rebuild everything with one script, as a master do-file, and end with one regression table, as `regress` and `esttab`

# From last time

Answer on your own first, then compare with a neighbour

1. You merge workers onto firms with `validate="m:1"` and pandas stops with a `MergeError`. What does that tell you?
2. Write the line that gives each person's income in the year before, in a panel sorted by person and year.

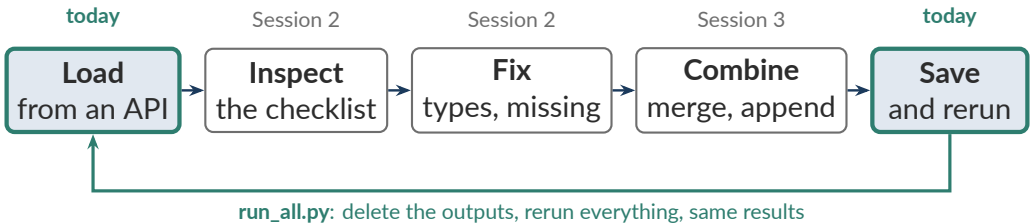
# From last time

Answer on your own first, then compare with a neighbour

1. You merge workers onto firms with `validate="m:1"` and pandas stops with a `MergeError`. What does that tell you?
   Answer: The firm key is not unique in the firm file: some firm appears twice. Find the duplicates before you merge
2. Write the line that gives each person's income in the year before, in a panel sorted by person and year.
   Answer: `df.groupby("pid")["inc"].shift(1)`. Today: `pct_change(4)`

# Every data task follows one recipe

Today: load from an API, then save and rerun (everything)




- Today's two errors never crash a script: an API answer that stops after the first page, and a quarter built from two months.

# Today in Stata and in Python

| Task | Stata | Python |
|---|---|---|
| get help | `help regress` | `smf.ols?`, or hover in VS Code |
| save a download | `import ...`, `save` | `requests.post`, `write_text` |
| text to dates | `gen m = monthly(month, "YM")` | `pd.to_datetime(month, ...)` |
| months to quarters | `collapse (mean) cpi, by(q)` | `resample("QE").mean()` |
| yearly growth | `L4.cpi` after `tsset` | `pct_change(4)` |
| write a step once | `program define` | `def ...: return ...` |
| rebuild everything | a master do-file | `run_all.py` |
| regression, table | `regress`, `esttab` | `smf.ols(...).fit()`, `summary_col` |

- Two new words: an API is a web address a program asks for data; JSON is the text it answers in. Both come next.

# Getting help, as Stata's `help`

1. In the Interactive window: type `pd.to_datetime?` and press Shift+Enter. You get the help text, with every option, as `help monthly()`
2. In VS Code: hold the mouse over a name in your script (`resample`, `pct_change`). A box says what it does and which options it takes
3. On the web: search "pandas resample" or "statsmodels ols". The package's own pages are the manual, as Stata's PDF manuals
4. The course tutor prompt: paste one line and ask what each part does. Then check it, as you would check a classmate

- A name you do not know is one `?` away. Ask before you guess.

# What an API answers: JSON

**Stata:** `a codebook and the data, in one text file`

```
{"title": "Consumer Price Index (CPI) ...",
 "variables": [
    {"code": "ContentsCode", "values": ["00000808", "00000807", ...]},
    {"code": "Tid", "values": ["1980M01", "1980M02", ...]} ]}
```

```
{"data": [ {"key": ["1980M01"], "values": ["28.38"]},
           {"key": ["1980M02"], "values": ["28.81"]}, ... ]}
```

| | |
|---|---|
| `{ }` | a dictionary: names and what they hold, as variables and their values |
| `[ ]` | a list: items in order. Python counts from 0: `[0]` is the first, `[1]` the second, `[-1]` the last (Stata: `[1]`, `[2]`, `[_N]`) |
| `"Tid"` | Swedish for time: the list of months, the second variable, so `["variables"][1]` |

- The first answer describes the table (the metadata); the second holds the data. Every number arrives as text, in a list: `["28.38"]`.

# Get data from an API

# Steps 1–6: from SCB's answer to inflation

| | Question | Stata you know |
|---|---|---|
| 1 | what does the table hold? | `describe`, before you load |
| 2 | how do we fetch the data, and keep a copy? | `import`, then `save` in `raw/` |
| 3 | did every month arrive? | `assert _N == ...` |
| 4 | how do month labels become dates? | `monthly()` |
| 5 | how do months become quarters? | `collapse (mean) ..., by(q)` |
| 6 | how much did prices rise over the year? | `L4.` after `tsset` |

- Every step ends with a count: how many rows the source says exist, and how many we have.

# Now I'll show you in VS Code

Cells SETUP (run it, do not type it), S1, S2, S3: Steps 1–3

# Step 1: what does the table hold?

**Stata:** `describe using cpi` (what a file holds, before you load it)

```
r = requests.get(URL, timeout=30)           # ask SCB
r.raise_for_status()                        # stop if it said no
meta = r.json()                             # read the JSON answer
months = meta["variables"][1]["values"]     # 2nd variable: Tid
print(len(months), months[0], months[-1])   # count, first, last
```

```
560 1980M01 2026M08
```

| | |
|---|---|
| `URL` | SCB's web address for the CPI table, stored as text in the SETUP cell |
| `requests` | the package that asks a web server for something (`get`: ask) |
| `r`, `meta`, `months` | names we chose. `r` holds SCB's answer; `r.json()` reads it, `r.text` is the raw text |
| `len()` | how many items: `_N` |

- SCB says the table holds 560 months. Run it today and you get one or two more: SCB adds a month every month.

# Step 2: how do we fetch the data, and keep a copy?

**Stata:** `import ... using "https://..." · save data/raw/cpi`

```
r = requests.post(URL, json=QUERY, timeout=30)
r.raise_for_status()                  # stop if it said no
path = RAW / "cpi.json"               # data/raw/scb/cpi.json
path.write_text(r.text)               # save it as it came
rows = r.json()["data"]               # into memory, not a file
print(rows[0], rows[-1], sep="\n")    # first and last row
```

```
{'key': ['1980M01'], 'values': ['28.38']}
{'key': ['2026M08'], 'values': ['124.85']}
```

| | |
|---|---|
| `QUERY`, `RAW` | set in SETUP: what we ask for (CPI, every month); the folder `data/raw/scb` |
| `RAW / "cpi.json"` | `/` joins folder and file name; `write_text` writes the text into it |

- Save the raw answer first: SCB revises its numbers.

# Step 3: did every month arrive?

**Stata:** `assert _N == 560` (561 if you run it today)

```
print(len(rows), len(months))  # rows we got; months SCB lists
assert len(rows) == len(months), "months missing"
```

```
560 560
```

`assert` works as in Stata: if the two numbers ever differ, the script stops, with the message after the comma, “months missing”.

- Compare what you got with what the source says exists. This is the check people skip, because a short answer raises no error.

# A second API: the World Bank, which answers in pages

- We ask the World Bank for GDP per person for five countries: Sweden, Finland, Norway, Denmark and Iceland, every year 1960–2025
- 5 countries $\times$ 66 years = 330 rows
- The World Bank sends at most 50 rows per answer: a page. You ask for page 1, then page 2, and so on
- Each row is one country in one year, with its code: `SWE`, `FIN`, `NOR`, `DNK`, `ISL`

- So page 1 holds 50 of the 330 rows. Which 50?

# World Bank, page 1: which countries are on it?

Your bet first: hands up. The answer is on the next slide.

A. All five, about ten rows each
B. All five, the newest years only
C. One country only

# Now I'll show you in VS Code

Cells S4 to S7: the World Bank's page 1 (the bet's answer), Steps 4–6

# Does page 1 hold everything? The World Bank

```
r = requests.get(WB, params={"format": "json"})   # page 1
meta_wb, page1 = r.json()                         # the info; the rows
print(meta_wb["total"], meta_wb["pages"])  # rows; pages
print(len(page1), page1[0]["countryiso3code"],
      page1[-1]["countryiso3code"])               # first, last country
```

```
330 7
50 DNK DNK
```

| | |
|---|---|
| `WB` | the World Bank's address for the five countries' GDP per person (SETUP) |
| `meta_wb, page1 = ...` | the answer has two parts, the info and the rows; we name both |
| `["countryiso3code"]` | a row's country code, as `SWE` |

- 330 rows exist, in 7 pages. Page 1 holds 50, and all 50 are Denmark. A script that reads only page 1 raises no error.

# Dates: months, quarters, growth

# Step 4: how do month labels become dates?

**Stata:** `gen m = monthly(month, "YM") · destring cpi, replace`

```
cpi = pd.DataFrame(rows)         # rows -> a dataset
month = cpi["key"].str[-1]       # "2026M08"
cpi["date"] = pd.to_datetime(month, format="%YM%m")  # monthly()
cpi["cpi"] = pd.to_numeric(cpi["values"].str[0])     # destring
cpi[["date", "cpi"]].tail(2)     # list date cpi in -2/l
```

```
          date     cpi
558 2026-07-01  125.17
559 2026-08-01  124.85
```

`pd.DataFrame(rows)` — a dataset made from `rows`, our list from Step 2 (`pd` = pandas)
`.str[-1]`, `.str[0]` — each cell holds a list, `['2026M08']`: take its last or first item
`format="%YM%m"` — the mask, as `"YM"`: `%Y` a year, then the letter M, then `%m` a month.
`"2026"+"M"+"08"`

- Now pandas knows these are dates and numbers.

# Step 5: how do months become quarters?

**Stata:** `collapse (mean) cpi (count) months=cpi, by(q)`

```
cpi = cpi.set_index("date")["cpi"]          # tsset date; keep cpi
q = cpi.resample("QE").mean()   # collapse (mean) cpi, by(quarter)
n = cpi.resample("QE").count()   # (count) months=cpi, by(quarter)
pd.DataFrame({"cpi": q, "months": n}).tail(3).round(2)  # list
```

| | cpi | months |
|---|---|---|
| date | | |
| 2026-03-31 | 124.84 | 3 |
| 2026-06-30 | 124.84 | 3 |
| 2026-09-30 | 125.01 | 2 |

`set_index("date")` — the dates become the row labels, as `tsset`

`resample("QE")` — regroup the months into quarters. `QE`: quarter, labelled by its end date, so 2026Q1 shows as 2026-03-31

- Mean for an index or a rate, sum for a flow. The last quarter has 2 months.

# Step 6: how much did prices rise over the year?

**Stata:** `keep if months == 3` · `tsset q` · `gen infl = 100*(cpi/L4.cpi - 1)`

```
q = q[n == 3]                  # keep if months == 3
infl = q.pct_change(4) * 100   # gen infl = 100*(cpi/L4.cpi - 1)
infl.tail(3).round(1)          # list in -3/l
```

```
date
2025-12-31    0.5
2026-03-31    0.5
2026-06-30    0.4
Freq: QE-DEC, Name: cpi, dtype: float64
```

`q[n == 3]`: complete quarters only. `pct_change(4)`: against the same quarter a year before, so seasons cancel.

- Prices rose 0.4 per cent over the year to the second quarter of 2026.

# Check your understanding

Answer on your own first, then compare with a neighbour

Predict: (a) you turn monthly *job ads* into quarters with `resample("QE").mean()`. What is wrong? (b) An old answer online writes `resample("Q")`. What happens in pandas 3?

# Check your understanding

Answer on your own first, then compare with a neighbour

Predict: (a) you turn monthly *job ads* into quarters with `resample("QE").mean()`. What is wrong? (b) An old answer online writes `resample("Q")`. What happens in pandas 3?

**Answer:** (a) Ads are a flow: the quarter's number is the *sum* of three months; the mean is a third of it. For a price index or a rate, the mean is right. (b) It stops: `ValueError: Invalid frequency: Q ... Please use 'QE' instead`.

# Scale up: functions, loops, run_all

# Steps 7–10: from one file to one table

| | Question | Stata you know |
|---|---|---|
| 7 | how do we write a repeated step once? | `program define` |
| 8 | how do we read every file the same way? | `foreach` (Session 3) |
| | how do we rebuild everything, and check it? | a master do-file with `log using` |
| 9 | does inflation move with unemployment? | `regress infl unemp` |
| 10 | how do we save the table for a paper? | `esttab` |

- Every step answers one question. Say the question before you write the code.

# Now I'll show you in VS Code

Cells F0, S8 to S11, S13 to S15: a function, Steps 7–10, run_all and the table

(before S10: delete `data/processed` and `output`; skip S12, DuckDB)

# New Python: a function, as Stata's `program`

**Stata**

```
program define pct
    args x
    display 100 * `x'
end
pct 0.25
```

**Python**

```
def pct(x):
    return 100 * x

pct(0.25)
```

```
25.0
```

`def pct(x):`
the colon and the indent

the name and the input, like `program define` and `args`

`return`
the body, like the lines up to Stata's `end`

hands the result back, so you can save it: `y = pct(0.25)`

# Step 7: how do we write a repeated step once?

**Stata:** `program define read_scb` (Steps 2 and 4 inside, then `end`)

```
def read_scb(path):                      # program define; args path
    text = path.read_text()              # the file, as text
    answer = json.loads(text)            # read the JSON
    d = pd.DataFrame(answer["data"]) # its rows -> a dataset
    month = d["key"].str[-1]             # "2026M08"
    d.index = pd.to_datetime(month, format="%YM%m")  # monthly()
    values = d["values"].str[0]          # "124.85"
    return pd.to_numeric(values)         # destring; hand it back
```

Running the cell prints nothing: it only *defines* the function.

- Write a step once, name it, and test it. A fix in the function fixes every file that uses it.

The lab's version adds `encoding="utf-8"` to `read_text`, for Windows.

# Step 8: how do we read every file the same way?

**Stata:** `foreach f in cpi unemp { ... }` (the loop from Session 3)

```
series = {}                                  # an empty dictionary
for path in sorted(RAW.glob("*.json")):      # foreach file
    series[path.stem] = read_scb(path)       # box "cpi", box "unemp"
monthly = pd.concat(series, axis=1)          # side by side, on date
monthly.tail(2)                              # list in -2/l
```

```
                cpi  unemp
key
2026-07-01  125.17    8.6
2026-08-01  124.85    8.9
```

| | |
|---|---|
| `series = {}` | a dictionary, named boxes: then `series["cpi"]`, `series["unemp"]` |
| `glob`, `stem` | every `.json` file, sorted; `stem`: the name without `.json` |
| `concat(..., axis=1)` | side by side on the date, as `merge 1:1` (`axis=0`: append) |

- Two files today, fifty tomorrow: the loop does not change.

# How do we rebuild everything? `run_all.py`

**Python: the list in run_all.py**

```
SCRIPTS = ["scripts/01_get_data.py",
           "scripts/02_clean.py",
           "scripts/03_table.py"]
```

**Stata: master.do**

```
log using run_log, replace
do 01_get_data.do
do 02_clean.do
do 03_table.do
```

**Typed in the terminal, then what each script prints**

```
$ uv run run_all.py
01_get_data.py: cpi 560/560 months, unemp 308/308 months (SCB API)
02_clean.py: 868 monthly values in, 102 quarters out (2001Q1-2026Q2)
  dropped 84 of 186 quarters with a gap (84 before the unemployment data)
03_table.py: 102 quarters in, 2 models out (output/tables/regression.*)
run_all done: log in output/logs/run_log.txt
```

- `run_all.py` runs each script in order, stops at the first error, and saves every printed line in the log: rows in, rows out, and why.

# The delete-outputs test

Gentzkow and Shapiro (2014), *Code and Data for the Social Sciences*, ch. 2

1. Delete `data/processed/` and `output/`: everything the code made
2. Run `run_all.py` once
3. Everything comes back, with the same numbers; the log in `output/logs/run_log.txt` shows rows in and out of every step

   *"The proof in the pudding is that we can now delete all of the output files in the directory [...] and reproduce them by running `rundirectory.bat`. This is the precise sense in which the output is now replicable."*

– The log is the first thing to read when a number looks strange: a step that drops half the sample shows there before it shows in the regression.

# One table at the end of the pipeline

# Step 9: does inflation move with unemployment?

**Stata:** `use quarterly, clear · regress infl unemp`

```
q = pd.read_parquet("data/processed/quarterly.parquet")  # use
m1 = smf.ols("infl ~ unemp", data=q).fit()  # regress infl unemp
m1.params.round(2)                          # the coefficients
```

```
Intercept    6.46
unemp       -0.61
```

| | |
|---|---|
| `read_parquet` | `use`: parquet is a data file, as `.dta`, saved by `02_clean.py` |
| `smf` | statsmodels' formula part, the regression package (SETUP: `import statsmodels.formula.api as smf`) |
| `"infl ~ unemp"` | read it "infl on unemp": y ~ x, the constant added, as `regress infl unemp` |
| `.fit()`, `m1` | `.fit()` runs it; `m1` keeps the results, as `estimates store m1` |

- A Phillips curve, 2001–2026, read from the file the pipeline saved.

# Step 9, continued: standard errors over time

**Stata:** `regress infl unemp, vce(robust)` (close; over time: `newey ..., lag(4)`)

```
m1 = smf.ols("infl ~ unemp", data=q).fit(
    cov_type="HAC", cov_kwds={"maxlags": 4})  # robust over time
m2 = smf.ols("infl ~ unemp + infl_lag1", data=q).fit(
    cov_type="HAC", cov_kwds={"maxlags": 4})  # adds L.infl
print(m1.bse["unemp"].round(2), m2.params["infl_lag1"].round(2))
```

```
0.18 0.93
```

| | |
|---|---|
| `cov_type="HAC"` | which standard errors, as `vce()`: robust, and also to errors correlated over time (Newey–West) |
| `cov_kwds={"maxlags": 4}` | the options for it, as a dictionary: up to 4 quarters back, as `lag(4)` |
| `infl_lag1` | last quarter's inflation, made in `02_clean.py`: `L.infl` |
| `m1.bse["unemp"]` | a standard error; `m2.params`: the coefficients |

- Same coefficients, honest standard errors: unemployment's SE is 0.18. In model 2 last quarter's inflation takes over (0.93).

# Step 10: how do we save the table for a paper?

**Stata:** `esttab m1 m2 using regression.csv, se star`

```
t = summary_col([m1, m2], stars=True,    # esttab m1 m2, star
                float_format="%.2f")     # b(%9.2f) se(%9.2f)
t.tables[0].loc["N"] = [str(int(m1.nobs)), str(int(m2.nobs))]
t.tables[0].to_csv(TABLES / "regression.csv")  # using .csv
tex_file = TABLES / "regression.tex"
tex_file.write_text(t.as_latex())              # and using .tex
```

```
607
```

| | |
|---|---|
| `summary_col` | models side by side, as `esttab`; `t.tables[0]` is the table |
| `.loc["N"] = ...` | adds a row N: each model's number of observations (`nobs`) |
| `TABLES` | the folder `output/tables` (SETUP). The 607: characters written |

- The CSV opens in Excel and Word; the `.tex` goes straight into a paper. Never copy numbers from the screen.

# The table, as the pipeline saved it

```
================================
                 infl I  infl II
--------------------------------
Intercept       6.46***  0.98
                (1.35)   (0.79)
unemp           -0.61*** -0.11
                (0.18)   (0.10)
infl_lag1                0.93***
                         (0.09)
R-squared       0.06     0.88
R-squared Adj.  0.05     0.88
N               102      102
================================
```

- Model I: a quarter with one point higher unemployment had 0.6 points lower inflation, on average
- Model II: most of that is last quarter's inflation carrying over (0.93); the unemployment coefficient falls to $-0.11$
- A correlation over time, not a causal effect

- N = 102 is the number in the log. The table and the log must agree.

# For the exam period: predict, test, explain

1. Predict before you run: write down what the line prints. The exam asks exactly that, in WISEflow, without Python and without AI
2. Test on a case whose answer you know: count rows in and out, `assert` what must hold, compute one number by hand
3. Explain every line in one sentence, including the lines an AI wrote. If you cannot, you do not own the result yet

– Code that runs is not code that is right. Code you can predict, test and explain is.

Practise: the sample exam and the practice bank on the course site, without AI.

# Today's lab: a mini-pipeline

1. Unzip `s4-lab.zip` into `na446a-starter`; open `s4-lab/scripts/01_get_data.py`; run a script whole with ▷
2. In class: TODO 1, then 2a–2d in `02_clean.py`. Each TODO shows the Stata line; you write the Python
3. If time: review an AI's pipeline, `ai_02_clean.py`: three `asserts` that catch it, then fix it
4. At home: TODOs 3–5, then `check_s4.py` (it does the delete-outputs test)

**Now I'll show you in VS Code**

- No network, or SCB answers 429 ("too many requests")? Set `USE_SAVED = True`: the saved copy runs the same way.

Never paste personal or register data into a chatbot. API keys (none today) go in `.env`, never in code.

# Summary

1. An API: ask for the metadata, ask for the data, save the raw answer, `assert` the count; an answer in pages needs a loop to the last page
2. Dates: `to_datetime` with a format; `resample("QE")` with a rule and a count; `pct_change(4)`; one function, one loop over the files
3. `run_all.py` rebuilds everything and logs rows in and out, as a master do-file; `summary_col` gives one table as CSV and LaTeX, as `esttab`

**Self-study:** DuckDB, which reads only what a query needs, so a file need not fit in memory (appendix).
**Before the exam:** the World Bank exercise and its self-check (about 90 minutes; solution after three days), then the sample exam, without AI.

# Test yourself before next time

Without AI: this is what you should manage on your own

1. The metadata lists 308 months; your data have 300 rows. Name two likely causes and the line that stops your script.
2. Write the line that turns labels such as `"2024M07"` into dates.
3. Monthly hours worked to quarters: which rule, and why? And for the unemployment rate?
4. A function lacks `return`: what does it give back, and `pd.concat` then?
5. Write the SQL that gives the mean of `infl` per year from `quarterly.parquet`.
6. What is the delete-outputs test, and what does it prove?
7. A script reads only the World Bank's page 1. What does it report, and what stops it?

# Appendix

# Every page, with a loop (the exercise)

```
meta, rows = get_page(name, 1)  # page 1, and the info
# forvalues p = 2/`pages'
for page in range(2, meta["pages"] + 1):
    rows = rows + get_page(name, page)[1]  # append it
# assert _N == total
assert len(rows) == meta["total"], "rows missing"
```

- Page 1 says how many pages exist; the loop asks for the rest
- `get_page` (given in the exercise) sends `page=...` with the request and saves each page in `data/raw/wb/`
- Last year's course code set `per_page=130` for 132 rows and silently lost two years of Swedish data: a larger page is not a check

# Self-study: DuckDB, for a file too large to load

```
duckdb.sql("""
  SELECT year(date) AS year, AVG(infl) AS infl
  FROM 'data/processed/quarterly.parquet'
  WHERE year(date) >= 2024
  GROUP BY year ORDER BY year""").df().round(2)
```

```
   year  infl
0  2024  2.86
1  2025  0.68
2  2026  0.47
```

- DuckDB reads only the rows and columns it needs, so the file need not fit in memory.

SQL: the language of databases. Stata has nothing like it: `use` loads the whole file.

# API keys go in `.env`, never in code

```
from dotenv import load_dotenv
import os                          # talks to the computer
load_dotenv()                    # reads .env
key = os.getenv("FRED_API_KEY")  # the key, never in the script
```

- `.env` is a text file in the project folder with one line: `FRED_API_KEY=...`
- `.gitignore` lists `.env`, so the key never reaches GitHub
- SCB and the World Bank need no key; FRED (US data) does